

Идентифицирующие материалы программы «Анализатор эффективности стратегий координации для групп роботизированных наземных транспортно-технологических машин (АЭСК-НТТМ)»

Соловьев В. В., i@viktor-solovev.ru

*Санкт-Петербургский государственный архитектурно-строительный
университет, Санкт-Петербург*

SPIN-код: 6187-7512, ORCID: 0009-0003-5573-8862

1 Введение

АЭСК-НТТМ предназначена для комплексного анализа результатов вычислительных экспериментов, проведенных в симуляторе многозадачной координации групп роботизированных наземных транспортно-технологических средств (СМК-НТТС) (Свидетельство о гос. регистрации программы для ЭВМ № 2025686591 от 6 ноября 2025 г.). Программа реализует математические методы оценки справедливости распределения нагрузки и эффективности различных стратегий координации в группе роботизированных наземных транспортно-технологических машин (РНТТМ). Программа реализована на языке Go.

Полный текст программного средства представлен в приложении¹.

2 Математический аппарат, реализованный в АЭСК-НТТМ

Для формализации введем следующие обозначения. Пусть в эксперименте участвует группа из N машин $\{R_1, \dots, R_N\}$. В ходе сценария выполняется S дискретных шагов моделирования. На каждом шаге s для каждой машины R_i вычисляются два показателя нагрузки:

¹ исходный код для самостоятельного проведения расчета или предложений для апробации доступен на https://github.com/dreddsa5dies/phd/Dissertation/code/src/cmd/stat_analyzer

Количество выполненных задач за шаг s . Обозначим через $\mathcal{T}_s(i)$ множество задач T_j , выполнение которых было завершено машиной R_i на шаге s . Тогда показатель определяется как

$$n_{i,s} = |\mathcal{T}_s(i)| \quad (1)$$

Суммарные энергетические затраты за шаг s . Пусть для каждой задачи T_j величина Θ_{T_j} выражает её сложность, выраженную в единицах потреблённого энергетического ресурса. Тогда суммарные затраты машины R_i на шаге s это

$$\varepsilon_{i,s} = \sum_{T_j \in \mathcal{T}_s(i)} \Theta_{T_j} \quad (2)$$

Таким образом, на каждом шаге s состояние группы РНТТМ описывается двумя векторами:

- вектор распределения количества задач: $\mathbf{n}_s = (n_{1,s}, n_{2,s}, \dots, n_{N,s})$,
- вектор распределения энергозатрат: $\boldsymbol{\varepsilon}_s = (\varepsilon_{1,s}, \varepsilon_{2,s}, \dots, \varepsilon_{N,s})$.

Для произвольного набора неотрицательных значений $\mathbf{x} = (x_1, x_2, \dots, x_N)$, характеризующих распределение ресурса (нагрузки) между N РНТТМ, коэффициент Джини $G(\mathbf{x})$ вычисляется по формуле:

$$G(\mathbf{x}) = \frac{\sum_{i=1}^N \sum_{j=1}^N |x_i - x_j|}{2N \sum_{i=1}^N x_i}, \quad \text{где } x_i \geq 0, \quad (3)$$

а свойства коэффициента $G(\mathbf{x}) = 0$ соответствует случаю **абсолютного равенства** ($x_1 = x_2 = \dots = x_N$), $G(\mathbf{x}) = 1$ соответствует случаю **максимального неравенства** (весь ресурс сконцентрирован у одного РНТТМ, а у остальных равен нулю).

Для каждого шага моделирования s вычисляются два конкретных коэффициента, отражающих справедливость распределения разных аспектов нагрузки.

Коэффициент Джини по количеству задач на шаге s :

$$G_s^{\text{задачи}} = G(\mathbf{n}_s) = \frac{\sum_{i=1}^N \sum_{j=1}^N |n_{i,s} - n_{j,s}|}{2N \sum_{i=1}^N n_{i,s}}. \quad (4)$$

Коэффициент Джини по энергозатратам на шаге s :

$$G_s^{\text{энергия}} = G(\boldsymbol{\varepsilon}_s) = \frac{\sum_{i=1}^N \sum_{j=1}^N |\varepsilon_{i,s} - \varepsilon_{j,s}|}{2N \sum_{i=1}^N \varepsilon_{i,s}}. \quad (5)$$

Для получения интегральных оценок эффективности стратегии координации Γ за весь эксперимент рассчитываются сводные статистики:

1. Среднее значение коэффициента по всем S шагам сценария:

$$\overline{G}^{\text{задачи}}(\Gamma) = \frac{1}{S} \sum_{s=1}^S G_s^{\text{задачи}}, \quad \overline{G}^{\text{энергия}}(\Gamma) = \frac{1}{S} \sum_{s=1}^S G_s^{\text{энергия}}. \quad (6)$$

2. Медианное значение коэффициента по всем шагам:

$$\text{med } G^{\text{задачи}}(\Gamma) = \text{median}(G_1^{\text{задачи}}, \dots, G_S^{\text{задачи}}), \quad (7)$$

$$\text{med } G^{\text{энергия}}(\Gamma) = \text{median}(G_1^{\text{энергия}}, \dots, G_S^{\text{энергия}}). \quad (8)$$

Одновременно с этим, пусть для стратегии координации Γ в эксперименте с общим количеством задач M и количеством машин N определены:

Процент выполнения задач:

$$P(\Gamma) = \frac{\sum_{s=1}^S n_s^{\text{выполнено}}}{M} \times 100\%, \quad (9)$$

где $n_s^{\text{выполнено}}$ – количество задач, выполненных на шаге s .

Средние энергетические затраты на одну задачу:

$$E_{\text{ср}}(\Gamma) = \frac{\sum_{s=1}^S \varepsilon_s}{\sum_{s=1}^S n_s^{\text{выполнено}}}, \quad (10)$$

где ε_s – общие энергозатраты на шаге s .

Общее время выполнения эксперимента:

$$T(\Gamma) = \sum_{s=1}^S \tau_s, \quad (11)$$

где τ_s – реальное время выполнения шага s .

Коэффициент энергоэффективности:

$$\eta(\Gamma) = \frac{\sum_{s=1}^S n_s^{\text{выполнено}}}{\sum_{s=1}^S \varepsilon_s} \times 1000. \quad (12)$$

Листинг 1: Код обрабатываемых метрик от СМК-НТТС

```
// Метрики
type Metrics struct {
    // Общее количество задач всего
    TotalTasks int 'json:"totalTasks"'
    // Общее количество машин всего
    TotalMachine int 'json:"totalMachine"'
    // Общая энергоемкость машин
    TotalEnergyMachines float64 'json:"totalEnergyMachines"'
    // Список машин
    AllMachines []models.Machine 'json:"allMachines"'
    // Общее энергоемкость задач
    TotalEnergyTasks float64 'json:"totalEnergyTasks"'
    // Список задач
    AllTasks []models.Task 'json:"allTasks"'
    // Общее время выполнения
    TotalTime string 'json:"totalTime"'
    // Метрики каждого шага
    StepMetric []StepMetrics 'json:"stepMetrics"'
}

type StepMetrics struct {
    Step int 'json:"step"' // Номер прогона
    // Количество выполненных задач
    LenTasksDone int 'json:"lenTasksDone"'
    // Перечень исполненных задач по ID
    TasksDone []string 'json:"tasksDone"'
    // Энергозатраты машин
    EnergyUsed float64 'json:"energyUsed"'
    // Не исполненные задачи
    LenNotExecTasks int 'json:"lenNotExecTasks"'
    // Реальное время выполнения шага
    RealTime string 'json:"realTime"'
    // детализация по машинам:
    // string1 -> machineID, string2 -> taskID, float64 -> потрачено
    MachineStats map[string]map[string]float64 'json:"machineStats"'
}
```

3 Входные данные

Входными данными является JSON файл (выходной отчет СМК-НТТС), который соответствует структуре в листинге 1.

4 Исходный код АЭСК-НТТМ

Листинг 1: файл stat_analyzer/main.go

```
package main

import (
    "encoding/json"
    "flag"
    "fmt"
    "log"
    "math"
    "os"
    "sort"
    "strings"

    "github.com/dreddsa5dies/phd/Dissertation/code/src/models"
    "github.com/dreddsa5dies/phd/Dissertation/code/src/strategies"
)

// Структуры для результатов
type (
    scenarioData map[string]strategies.Metrics // стратегия -> метрики
    allData      map[string]scenarioData      // сценарий -> scenarioData
)

// Результаты анализа
type giniAnalysisResult struct {
    Scenario string
    Strategy string
    Step     int

    // Векторы распределения
    TaskDistribution []int //  $n_i$  - количество задач на машине
    EnergyDistribution []float64 //  $\epsilon_i$  - энергозатраты на машине

    // Коэффициенты Джини для шага
    GiniTasks float64 // по задачам
    GiniEnergy float64 // по энергии
}

type summaryResult struct {
    Scenario string // наименование сценария
    Strategy string // наименование стратегии

    // Статистики по шагам
    GiniTasksValues []float64 // по задачам
    GiniEnergyValues []float64 // по энергии

    // Сводные статистики
    AvgGiniTasks float64
    MedGiniTasks float64
    AvgGiniEnergy float64
    MedGiniEnergy float64
}
```

```

}

// Функция вычисления коэффициента Джини
func calculateGini(values []float64) float64 {
55  n := len(values)
    if n == 0 {
        return 0
    }

60  // Сумма всех элементов
    sumX := 0.0
    for _, v := range values {
        sumX += v
    }

65  // Если сумма нулевая, коэффициент Джини = 0
    // равномерное( распределение)
    if sumX == 0 {
        return 0
    }

70  // Вычисление суммы модулей разностей
    sumAbsDiff := 0.0
    for i := 0; i < n; i++ {
        for j := 0; j < n; j++ {
            sumAbsDiff += math.Abs(values[i] - values[j])
        }
    }

80  // Формула коэффициента Джини
    gini := sumAbsDiff / (2 * float64(n) * sumX)

    // Ограничиваем значения [0, 1]
    if gini < 0 {
85        return 0
    }
    if gini > 1 {
        return 1
    }

90  return gini
}

// Анализ одного шага
func analyzeStep(scenario, strategy string, stepMetrics strategies.
    StepMetrics, allMachines []models.Machine) giniAnalysisResult {
95  result := giniAnalysisResult{
        Scenario: scenario,
        Strategy: strategy,
        Step:      stepMetrics.Step,
    }

100  machineCount := len(allMachines)

```

```

result.TaskDistribution = make([]int, machineCount)
result.EnergyDistribution = make([]float64, machineCount)

105 machineIndex := make(map[string]int, machineCount)
    for i, machine := range allMachines {
        machineIndex[machine.ID] = i
    }

110 // MachineStats: map[machineID]map[taskID]energy
    for machineID, taskEnergyMap := range stepMetrics.MachineStats {
        idx, exists := machineIndex[machineID]
        if !exists {
            log.Printf("Предупреждение: машина %s из MachineStats отсутствует
в AllMachines", machineID)
115             continue
        }

        // Количество задач = число записей в мапе задач
        result.TaskDistribution[idx] = len(taskEnergyMap)

120 // Суммарная энергия = сумма всех значений в мапе
        totalEnergy := 0.0
        for _, energy := range taskEnergyMap {
            totalEnergy += energy
125        }
        result.EnergyDistribution[idx] = totalEnergy
    }

    // Преобразуем в []float64 для calculateGini
130 taskValues := make([]float64, machineCount)
    for i, count := range result.TaskDistribution {
        taskValues[i] = float64(count)
    }
    energyValues := make([]float64, machineCount)
135 copy(energyValues, result.EnergyDistribution)

    result.GiniTasks = calculateGini(taskValues)
    result.GiniEnergy = calculateGini(energyValues)

140 return result
}

// Анализ всей стратегии
func analyzeStrategy(scenario, strategy string, metrics strategies.
Metrics) summaryResult {
145 result := summaryResult{
    Scenario: scenario,
    Strategy: strategy,
}

150 if flag.Lookup("verbose").Value.String() == "true" {
    totalTasks := 0

```

```

    for _, step := range metrics.StepMetric {
        totalTasks += step.LenTasksDone
    }
155   log.Printf("Сценарий %s, Стратегия %s: всего машин = %d, всего
задач = %d", scenario, strategy, len(metrics.AllMachines), totalTasks
)
}

// Анализируем каждый шаг
160   for _, stepMetrics := range metrics.StepMetric {
        stepResult := analyzeStep(scenario, strategy, stepMetrics, metrics.
AllMachines)
        result.GiniTasksValues = append(result.GiniTasksValues, stepResult.
GiniTasks)
        result.GiniEnergyValues = append(result.GiniEnergyValues, stepResult
.GiniEnergy)
    }

165   // Вычисляем статистики
    if len(result.GiniTasksValues) > 0 {
        result.AvgGiniTasks = mean(result.GiniTasksValues)
        result.MedGiniTasks = median(result.GiniTasksValues)
    }

170   if len(result.GiniEnergyValues) > 0 {
        result.AvgGiniEnergy = mean(result.GiniEnergyValues)
        result.MedGiniEnergy = median(result.GiniEnergyValues)
    }

175   return result
}

// Сбор всех результатов
180   func collectAllResults(allData allData) []summaryResult {
        var results []summaryResult

        for scenario, scenarioData := range allData {
            for strategy, metrics := range scenarioData {
185                result := analyzeStrategy(scenario, strategy, metrics)
                results = append(results, result)

                // Детальная информация для отладки опционально()
                log.Printf("Анализ завершен: Сценарий=%s, Стратегия=%s", scenario,
strategy)
190                log.Printf(" Коэффициент Джини по задачам: среднее=%.4f,
медиана=%.4f",
                    result.AvgGiniTasks, result.MedGiniTasks)
                log.Printf(" Коэффициент Джини по энергии: среднее=%.4f,
медиана=%.4f",
                    result.AvgGiniEnergy, result.MedGiniEnergy)
            }
195   }
}

```



```

    return results
}

200 // Сохранение в CSV
func saveSummaryCSV(results []summaryResult) {
    file, err := os.Create("gini_results.csv")
    if err != nil {
        log.Fatalf("Ошибка создания CSV файла: %v", err)
205    }
    defer file.Close()

    // Заголовок CSV
    header := "СценарийСтратегияСреднее;; значение задачи()Медиана;
        задачи()Среднее; значение энергия()Медиана; энергия()\n"
210    file.WriteString(header)

    // Данные
    for _, result := range results {
        line := fmt.Sprintf("%s;%s;%.6f;%.6f;%.6f;%.6f\n",
215            result.Scenario,
            result.Strategy,
            result.AvgGiniTasks,
            result.MedGiniTasks,
            result.AvgGiniEnergy,
220            result.MedGiniEnergy,
        )
        file.WriteString(line)
    }

225    log.Println("Результаты сохранены в gini_results.csv")
}

// Дополнительная функция для детального анализа по( шагам)
func saveDetailedAnalysisCSV(allData allData) {
230    file, err := os.Create("gini_detailed.csv")
    if err != nil {
        log.Fatalf("Ошибка создания детального CSV файла: %v", err)
    }
    defer file.Close()

235    header := "СценарийСтратегияШагДжини;;; задачи()Джини; энергия()\n"
    file.WriteString(header)

    for scenario, scenarioData := range allData {
240        for strategy, metrics := range scenarioData {
            for _, stepMetrics := range metrics.StepMetric {
                stepResult := analyzeStep(scenario, strategy, stepMetrics,
                    metrics.AllMachines)

                line := fmt.Sprintf("%s;%s;%d;%.6f;%.6f\n",
245                    stepResult.Scenario,

```

```

        stepResult.Strategy,
        stepResult.Step,
        stepResult.GiniTasks,
        stepResult.GiniEnergy,
250    )
        file.WriteString(line)
    }
}
}
255 log.Println("Детальные результаты сохранены в gini_detailed.csv")
}

// Основная функция
260 func main() {
    // Определение флагов
    filePath := flag.String("file", "report.json", "Путь к файлу report.
        json")
    showHelp := flag.Bool("help", false, "Показать справку")
    verbose := flag.Bool("verbose", false, "Подробный вывод")
265
    flag.Parse()

    // Вывод справки
    if *showHelp {
270        printHelp()
        return
    }

    // Проверка существования файла
275    if _, err := os.Stat(*filePath); os.IsNotExist(err) {
        log.Fatalf("Файл не найден: %s", *filePath)
    }

    if *verbose {
280        log.Printf("Начинаю анализ файла: %s", *filePath)
    }

    // Чтение файла
    b, err := os.ReadFile(*filePath)
285    if err != nil {
        log.Fatalf("Ошибка чтения %s: %v", *filePath, err)
    }

    if *verbose {
290        log.Printf("Файл прочитан успешно, размер: %d байт", len(b))
    }

    var all allData
    if err := json.Unmarshal(b, &all); err != nil {
295        log.Fatalf("Ошибка парсинга JSON: %v", err)
    }
}

```

```

300     if *verbose {
        log.Printf("JSON распарсен успешно")
        log.Printf("Найдено сценариев: %d", len(all))
        totalStrategies := 0
        for scenario, data := range all {
            totalStrategies += len(data)
            log.Printf("Сценарий '%s': %d стратегий", scenario, len(data))
305     }
        log.Printf("Всего стратегий: %d", totalStrategies)
    }

    // Сбор данных
310    results := collectAllResults(all)

    if *verbose {
        log.Printf("Анализ завершен, обработано %d результатов", len(results))
    }
315

    // Сохранение CSV
    saveSummaryCSV(results)
    saveDetailedAnalysisCSV(all)

320    fmt.Println("Анализ завершён. Файлы: gini_results.csv, gini_detailed.
        csv")

    // Дополнительная статистика
    if *verbose {
        printAdditionalStats(results)
325    }
}

// Функция для отображения справки
func printHelp() {
330    fmt.Println("Анализатор статистической справедливости распределения
        задач")
    fmt.Println("
        =====")
    fmt.Println()
    fmt.Println("Использование:")
    fmt.Println("  analyzer опции[]")
335    fmt.Println()
    fmt.Println("Опции:")
    fmt.Println("  --file string      Путь к файлу report.json по(
        умолчанию: report.json)")
    fmt.Println("  --help             Показать эту справку")
    fmt.Println("  --verbose          Подробный вывод процесса анализа")
340    fmt.Println()
    fmt.Println("Примеры:")
    fmt.Println("  analyzer --file=./data/report.json")
    fmt.Println("  analyzer --help")

```

```

fmt.Println()
345 fmt.Println("Выходные файлы:")
fmt.Println("  gini_results.csv    - сводные результаты по стратегиям"
)
fmt.Println("  gini_detailed.csv    - детальные результаты по шагам")
fmt.Println()
fmt.Println("Метрики расчета:")
350 fmt.Println("  Коэффициент Джини по количеству задач")
fmt.Println("  Коэффициент Джини по энергозатратам")
fmt.Println("  Среднее и медианное значения по всем шагам")
}

355 // Дополнительная функция для вывода статистики
func printAdditionalStats(results []summaryResult) {
    fmt.Println("\n *** СТАТИСТИЧЕСКАЯ СПРАВКА ***")

    // Группируем по сценариям
360 scenarios := make(map[string][]summaryResult)
    for _, r := range results {
        scenarios[r.Scenario] = append(scenarios[r.Scenario], r)
    }

365 for scenario, list := range scenarios {
    fmt.Printf("\nСценарийn: %s\n", scenario)
    // Заголовок с компактными метками
    fmt.Printf("%-25s %8s %8s %8s %8s\n",
        "Стратегия", "Срзад.", "Медзад.", "Срэн.", "Медэн.")
370 fmt.Println(strings.Repeat("-", 65))

    for _, result := range list {
        fmt.Printf("%-25s %8.4f %8.4f %8.4f %8.4f\n",
            result.Strategy,
375 result.AvgGiniTasks,
            result.MedGiniTasks,
            result.AvgGiniEnergy,
            result.MedGiniEnergy)
    }
380 }

    fmt.Println("\nИнтерпретацияn коэффициентов Джини:")
    fmt.Println("  -0.00.3: Высокая справедливость")
    fmt.Println("  -0.30.6: Умеренная неравномерность")
385 fmt.Println("  -0.60.8: Высокая неравномерность")
    fmt.Println("  -0.81.0: Экстремальная неравномерность")
}

// Вспомогательные функции статистики
390 func mean(values []float64) float64 {
    if len(values) == 0 {
        return 0
    }
    sum := 0.0

```

```

395   for _, v := range values {
        sum += v
    }
    return sum / float64(len(values))
}
400
func median(values []float64) float64 {
    if len(values) == 0 {
        return 0
    }
405
    // Создаем копию для сортировки
    sorted := make([]float64, len(values))
    copy(sorted, values)
    sort.Float64s(sorted)
410
    n := len(sorted)
    if n%2 == 0 {
        // Четное количество элементов
        return (sorted[n/2-1] + sorted[n/2]) / 2
415    }
    // Нечетное количество элементов
    return sorted[n/2]
}

```